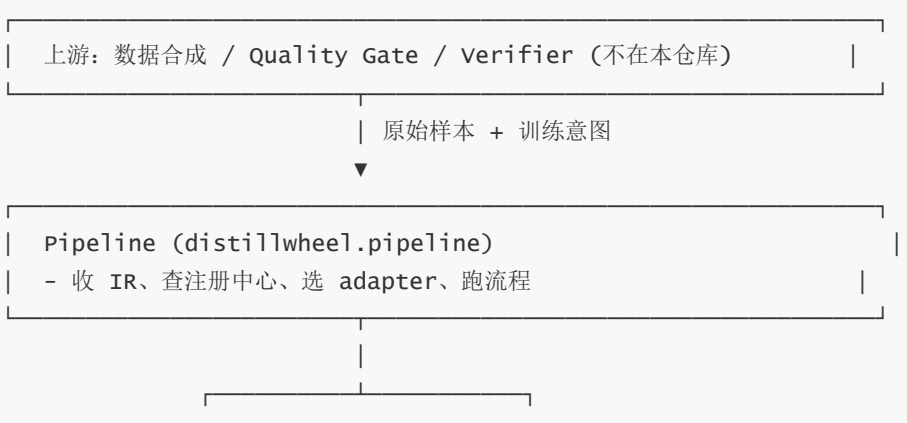
# DistillWheel 训练后端 Adapter 代码框架设计方案

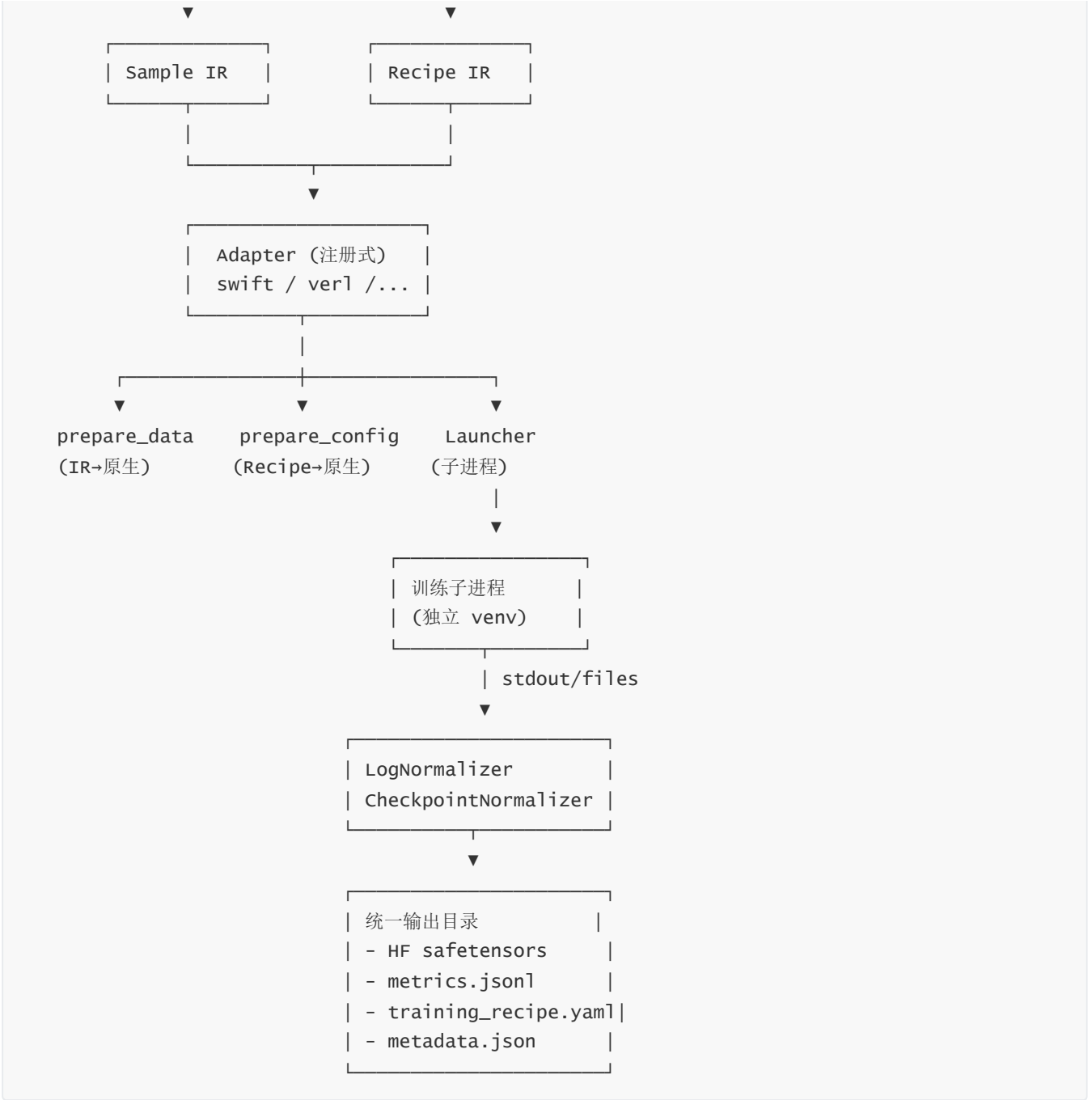
> 本文档对应 `DistillWheel.md` 的 TODO 项：数据层 IR、Recipe IR、Launcher 子进程隔离、Checkpoint 归一化、Log 归一化。
>
> **范围**：仅训练执行层。当前只接入 `swift` 和 `verl` 两个框架（RL/OPD 走 verl，其它训练方法走 swift），但通过注册机制保留扩展能力——新增框架只需写一个 adapter 包，不动主框架代码。

## 目录



## 一、整体架构






核心思想：**主进程只跑 IR 转换与编排，不 import 任何训练框架。**每个 adapter 通过 entry point 注册到一个全局注册表，Pipeline 按 `recipe.stage` + 路由策略选 adapter。

---

# 二、目录结构

```
distillwheel/
├── pyproject.toml
├── distillwheel/
│   ├── __init__.py
│   ├── core/                          # 框架无关核心
│   │   ├── __init__.py
│   │   ├── ir/
│   │   │   ├── sample.py              # Sample / Message 数据类
│   │   │   ├── recipe.py              # Recipe 数据类
```

```
|   |   |   ├── metric.py           # 归一化指标 schema
|   |   |   └── validators.py       # IR 校验逻辑
|   |   ├── adapter.py              # BackendAdapter 抽象基类
|   |   ├── launcher.py             # Launcher 抽象基类 + SubprocessLauncher
|   |   ├── checkpoint.py           # CheckpointNormalizer 基类
|   |   ├── logparser.py            # LogParser 基类 + 通用工具
|   |   ├── registry.py             # 全局注册中心
|   |   ├── router.py               # stage→adapter 路由策略
|   |   ├── envspec.py              # backend 环境声明
|   |   └── errors.py               # 统一异常类型
|   ├── pipeline/
|   |   ├── __init__.py
|   |   ├── orchestrator.py         # 主流程编排
|   |   ├── preflight.py            # dry-run 预检
|   |   └── artifacts.py            # 输出目录布局
|   ├── backends/                   # 各框架 adapter (独立子包)
|   |   ├── __init__.py
|   |   ├── swift/
|   |   |   ├── __init__.py         # register_adapter() 入口
|   |   |   ├── adapter.py
|   |   |   ├── data_writer.py      # IR → swift 数据格式
|   |   |   ├── recipe_mapper.py    # Recipe → swift 配置
|   |   |   ├── launcher.py
|   |   |   ├── checkpoint.py
|   |   |   ├── logparser.py
|   |   |   └── env.yaml            # 该 backend 的 venv 描述
|   |   └── verl/
|   |       ├── __init__.py
|   |       ├── adapter.py
|   |       ├── data_writer.py      # IR → parquet
|   |       ├── recipe_mapper.py
|   |       ├── launcher.py         # Ray job 启动
|   |       ├── checkpoint.py       # FSDP 分片 → HF merge
|   |       ├── logparser.py
|   |       └── env.yaml
|   ├── cli.py                      # `distillwheel run --recipe xxx.yaml`
|   └── version.py
├── tests/
|   ├── unit/
|   └── e2e/
└── docs/
    └── add_new_backend.md
```

**要点**:

- `core/` 是纯 Python、零训练框架依赖，能在任何环境装。
- `backends/<name>/` 每个子包**只在自己的 venv 里被子进程 import**，主进程只 import `adapter.py` 顶层（不能在顶层 import torch/trl/verl）。
- 通过 `entry_points` 注册，新 backend 不需要改 `core` 或 `pipeline`。

---

# 三、核心抽象层

## 3.1 数据层 IR

`distillwheel/core/ir/sample.py`：

```python
from dataclasses import dataclass, field
from typing import Literal, Optional, Union

TaskType = Literal["sft", "preference", "kto", "rl_prompt"]

@dataclass
class Message:
    role: Literal["system", "user", "assistant", "tool"]
    content: str
    tool_calls: Optional[list[dict]] = None
    tool_call_id: Optional[str] = None

@dataclass
class Sample:
    id: str
    task_type: TaskType
    messages: Optional[list[Message]] = None
    prompt: Union[str, list[Message], None] = None
    chosen: Union[str, list[Message], None] = None
    rejected: Union[str, list[Message], None] = None
    completion: Optional[str] = None
    label: Union[bool, float, None] = None
    tools: Optional[list[dict]] = None
    images: Optional[list[str]] = None
    meta: dict = field(default_factory=dict)

    def validate(self) -> None:
        """根据 task_type 校验必填字段。详见 validators.py"""
        ...
```

`distillwheel/core/ir/validators.py` 负责按 `task_type` 检查字段组合（例如 `preference` 必须有 `prompt + chosen + rejected`）。**IR 层只存原文**，不 tokenize、不 apply chat template。

**流式 I/O**：定义 `SampleStream` 接口（`Iterable[Sample]`），所有 adapter 的 `prepare_data` 都接受流而非 list，避免大数据集 OOM。

```python
# core/ir/sample.py
from typing import Iterable, Protocol

class SampleStream(Protocol):
    def __iter__(self) -> Iterable[Sample]: ...
```

## 3.2 Recipe IR

`distillwheel/core/ir/recipe.py`:

```python
from dataclasses import dataclass, field
from typing import Literal, Optional

Stage = Literal["sft", "dpo", "kto", "grpo", "ppo", "rloo", "opd"]

@dataclass
class PEFTConfig:
    type: Literal["lora", "qlora", "full"]
    r: int = 16
    alpha: int = 32
    target_modules: list[str] = field(default_factory=list)
    dropout: float = 0.0

@dataclass
class OptimConfig:
    lr: float = 5e-5
    scheduler: str = "cosine"
    warmup_ratio: float = 0.03
    weight_decay: float = 0.0

@dataclass
class TrainConfig:
    epochs: float = 1.0
    global_batch: int = 32
    micro_batch: int = 1
    grad_accum: int = 1
    max_len: int = 4096
    seed: int = 42

@dataclass
class ParallelConfig:
    dp: int = 1
    tp: int = 1
    pp: int = 1
    zero_stage: int = 0

@dataclass
class RLConfig:
    kl_coef: float = 0.05
    clip: float = 0.2
    rollout_n: int = 4
    rollout_engine: Literal["vllm", "sglang"] = "vllm"
    reward_fn_ref: Optional[str] = None  # "module.path:fn_name"

@dataclass
class IOConfig:
    output_dir: str
    save_steps: int = 500
    logging_steps: int = 10
```

```
    resume_from: Optional[str] = None

@dataclass
class Recipe:
    stage: Stage
    base_model: str
    train: TrainConfig
    optim: OptimConfig
    io: IOConfig
    peft: Optional[PEFTConfig] = None
    precision: Literal["bf16", "fp16", "fp8"] = "bf16"
    parallel: ParallelConfig = field(default_factory=ParallelConfig)
    rl: Optional[RLConfig] = None
    target_template: Optional[str] = None     # 强制 chat template 名称
    backend_hint: Optional[str] = None        # 用户可强制选 backend
    meta: dict = field(default_factory=dict)

    @classmethod
    def from_yaml(cls, path: str) -> "Recipe": ...
    def to_yaml(self, path: str) -> None: ...
```

**版本号策略**：Recipe 序列化时附带 `_recipe_schema_version`，每次 schema 变动 bump 版本，pipeline 启动时校验。

## 3.3 Adapter 基类

`distillwheel/core/adapter.py`：

```
from abc import ABC, abstractmethod
from pathlib import Path
from .ir.sample import SampleStream
from .ir.recipe import Recipe
from .launcher import Launcher
from .checkpoint import CheckpointNormalizer
from .logparser import LogParser
from .envspec import EnvSpec

class BackendAdapter(ABC):
    """单个训练框架的接入点。所有方法都在主进程执行，
    禁止在模块顶层或这些方法里 import 该框架本身。
    框架代码只在子进程通过 Launcher 启动时才被加载。"""

    name: str                                 # 唯一标识 ("swift", "verl")
    supported_stages: tuple[str, ...]         # 支持的 stage 列表
    env_spec: EnvSpec                         # venv / 镜像声明

    @abstractmethod
    def prepare_data(
        self,
        stream: SampleStream,
        recipe: Recipe,
        workdir: Path,
```

```
    ) -> Path:
        """把 IR 流式落盘成框架原生格式 (JSONL / parquet 等)，返回数据文件路径。"""

    @abstractmethod
    def prepare_config(
        self,
        recipe: Recipe,
        data_path: Path,
        workdir: Path,
    ) -> Path:
        """把 Recipe 翻译成框架原生配置 (YAML / argparse list)，返回配置文件路径。
        同时把原始 recipe.yaml 一并写入 workdir 用于可复现。"""

    @abstractmethod
    def build_launcher(
        self,
        config_path: Path,
        recipe: Recipe,
        workdir: Path,
    ) -> Launcher:
        """构造执行训练的 Launcher 实例。"""

    @abstractmethod
    def checkpoint_normalizer(self) -> CheckpointNormalizer:
        """返回该框架的 checkpoint 归一化器。"""

    @abstractmethod
    def log_parser(self) -> LogParser:
        """返回该框架的日志解析器。"""

    def supports(self, recipe: Recipe) -> bool:
        """默认按 stage 匹配；adapter 可重写做更细判断。"""
        return recipe.stage in self.supported_stages
```

## 3.4 Launcher 基类

`distillwheel/core/launcher.py`：

```
import subprocess
from abc import ABC, abstractmethod
from dataclasses import dataclass
from pathlib import Path
from typing import Iterator, Optional

@dataclass
class LaunchResult:
    returncode: int
    artifacts_dir: Path
    duration_s: float
    last_error: Optional[str] = None

class Launcher(ABC):
```

```
    """训练子进程的统一接口。所有 backend 必须用子进程方式启动，
    禁止 in-process 调用——避免版本冲突、崩溃传染、GPU 状态污染。"""

    @abstractmethod
    def prepare_env(self) -> None:
        """venv/镜像就绪性检查；准备工作目录、环境变量。"""

    @abstractmethod
    def command(self) -> list[str]:
        """完整命令行（含解释器路径，使用该 backend 自己的 venv python）。"""

    @abstractmethod
    def env(self) -> dict[str, str]:
        """传给子进程的环境变量。"""

    def launch(self) -> Iterator[str]:
        """启动子进程，按行 yield stdout/stderr。默认实现已够用，子类一般不重写。"""
        proc = subprocess.Popen(
            self.command(),
            env=self.env(),
            stdout=subprocess.PIPE,
            stderr=subprocess.STDOUT,
            text=True,
            bufsize=1,
        )
        try:
            assert proc.stdout is not None
            for line in proc.stdout:
                yield line.rstrip("\n")
        finally:
            proc.wait()
            self._returncode = proc.returncode

    @abstractmethod
    def collect_artifacts(self) -> Path:
        """训练结束后返回框架原生输出目录（供 CheckpointNormalizer 处理）。"""

    @property
    def returncode(self) -> int:
        return getattr(self, "_returncode", -1)
```

**子进程隔离三件套**：

1. **独立 venv**：每个 backend 的 `env.yaml` 声明 `python_version`、`pip_packages`，由 `tools/setup_backend_envs.py` 一次性创建。命令行用 `<backend_venv>/bin/python` 而非主进程 python。
2. **环境变量白名单**：默认不继承主进程的 `PYTHONPATH`、`LD_LIBRARY_PATH`，避免污染；只透传 `CUDA_VISIBLE_DEVICES`、`HF_HOME`、`WANDB_*` 等显式白名单字段。
3. **超时与心跳**：Pipeline 层包裹 `launch()` 迭代器，若 `N 分钟没新 stdout` 触发 `NCCL hang` 怀疑，写入诊断日志并杀子进程。

## 3.5 Checkpoint 归一化

`distillwheel/core/checkpoint.py`：

```
from abc import ABC, abstractmethod
from pathlib import Path
from dataclasses import dataclass

@dataclass
class NormalizedCheckpoint:
    final_dir: Path          # 最终模型 (HF safetensors)
    is_lora: bool
    step: Optional[int]
    base_model: str
    framework: str
    extra: dict              # 附加元数据 (recipe hash, commit, ...)

class CheckpointNormalizer(ABC):
    """把框架原生 checkpoint 转成统一 HF 目录布局。"""

    @abstractmethod
    def normalize(
        self,
        native_dir: Path,
        output_dir: Path,
        recipe_yaml_path: Path,
    ) -> NormalizedCheckpoint:
        """
        统一输出布局:
          output_dir/final/
            config.json
            tokenizer.json / tokenizer_config.json / special_tokens_map.json
            model.safetensors                (全参) 或 adapter_model.safetensors (LoRA)
            adapter_config.json              (LoRA only)
            training_recipe.yaml             (来自 recipe_yaml_path 的副本)
            metadata.json                    (NormalizedCheckpoint 的 JSON 序列化)
          output_dir/checkpoints/
            step_{N}/...                     (中间 checkpoint，可选)
        """
```

各 backend 的子类负责实际转换：

- swift：基本上是改目录名/拷文件即可。
- verl：调用 `verl.scripts.model_merger` 把 FSDP 分片 merge 成单文件，再走通用路径；或 DeepSpeed 走 `zero_to_fp32.py`。

## 3.6 Log 归一化

`distillwheel/core/ir/metric.py`：

```python
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class Metric:
    step: int
    timestamp: float
    stage: str
    # 通用
    loss: Optional[float] = None
    learning_rate: Optional[float] = None
    grad_norm: Optional[float] = None
    epoch: Optional[float] = None
    # RL 专属（使用统一键名）
    reward_mean: Optional[float] = None
    reward_std: Optional[float] = None
    kl: Optional[float] = None
    policy_loss: Optional[float] = None
    value_loss: Optional[float] = None
    entropy: Optional[float] = None
    # 扩展
    extra: dict = field(default_factory=dict)
```

`distillwheel/core/logparser.py`：

```python
from abc import ABC, abstractmethod
from typing import Iterable, Iterator
from .ir.metric import Metric

class LogParser(ABC):
    """从子进程 stdout 行流里抽取 Metric。"""

    @abstractmethod
    def parse_line(self, line: str) -> Optional[Metric]:
        """单行 → 0 或 1 条 Metric。无法识别返回 None。"""

    def parse_stream(self, lines: Iterable[str]) -> Iterator[Metric]:
        for line in lines:
            m = self.parse_line(line)
            if m is not None:
                yield m
```

各 backend 实现自己的 `parse_line` 后，Pipeline 把结果写到 `{output_dir}/metrics.jsonl`，下游所有可视化只读这个文件。

**字段映射约定**（写入 `docs/metric_mapping.md` 一并维护）：

| 统一字段 | swift 原始键 | verl 原始键 |
| --- | --- | --- |
| loss | loss | actor/loss 或 critic/loss |
| kl | kl_div | actor/kl 或 approx_kl |
| reward_mean | - | reward/mean |
| policy_loss | - | actor/pg_loss |

## 3.7 注册中心

`distillwheel/core/registry.py`：

```
from typing import Type
from .adapter import BackendAdapter

_REGISTRY: dict[str, Type[BackendAdapter]] = {}

def register_adapter(cls: Type[BackendAdapter]) -> Type[BackendAdapter]:
    """装饰器形式注册。"""
    if cls.name in _REGISTRY:
        raise ValueError(f"adapter {cls.name} already registered")
    _REGISTRY[cls.name] = cls
    return cls

def get_adapter(name: str) -> BackendAdapter:
    if name not in _REGISTRY:
        raise KeyError(f"adapter {name} not registered. known={list(_REGISTRY)}")
    return _REGISTRY[name]()

def list_adapters() -> list[str]:
    return list(_REGISTRY)

def load_entry_points() -> None:
    """从 setuptools entry_points 'distillwheel.backends' 自动加载。"""
    from importlib.metadata import entry_points
    for ep in entry_points(group="distillwheel.backends"):
        ep.load()   # backend 包的 __init__ 应当调用 register_adapter
```

`distillwheel/core/router.py`：

```
from .ir.recipe import Recipe
from .registry import get_adapter, list_adapters
from .adapter import BackendAdapter

# 默认路由策略：stage → backend name
_DEFAULT_ROUTE = {
    "sft":  "swift",
    "dpo":  "swift",
    "kto":  "swift",
    "grpo": "verl",
```

```
    "ppo":  "verl",
    "rloo": "verl",
    "opd":  "verl",
}

def resolve(recipe: Recipe) -> BackendAdapter:
    """1. recipe.backend_hint 优先
       2. 否则查 _DEFAULT_ROUTE
       3. 都没有则尝试遍历注册表找第一个 supports() 的 adapter"""
    if recipe.backend_hint:
        return get_adapter(recipe.backend_hint)
    if recipe.stage in _DEFAULT_ROUTE:
        return get_adapter(_DEFAULT_ROUTE[recipe.stage])
    for name in list_adapters():
        ad = get_adapter(name)
        if ad.supports(recipe):
            return ad
    raise ValueError(f"no adapter supports stage={recipe.stage}")
```

`distillwheel/core/envspec.py`：

```
from dataclasses import dataclass
from pathlib import Path

@dataclass
class EnvSpec:
    """声明 backend 所需的隔离环境，由 tools/setup_backend_envs.py 消费。"""
    venv_path: Path                     # 已创建的 venv 根目录
    python_executable: Path             # venv 内 python 完整路径
    required_packages: list[str]        # pip install 列表（含版本）
    extra_pip_args: list[str] = ()      # 例如 --extra-index-url
    cuda_constraint: Optional[str] = None
    health_check_cmd: Optional[list[str]] = None   # 启动前 sanity check

    def is_ready(self) -> bool:
        return self.python_executable.exists()
```

---

# 四、Backend 实现层

## 4.1 swift adapter

`distillwheel/backends/swift/__init__.py`：

```
from distillwheel.core.registry import register_adapter
from .adapter import SwiftAdapter

register_adapter(SwiftAdapter)
```

`distillwheel/backends/swift/adapter.py`（**主进程能 import，不能在顶层 import swift**）：

```python
from pathlib import Path
from distillwheel.core.adapter import BackendAdapter
from distillwheel.core.envspec import EnvSpec
from .data_writer import write_swift_jsonl
from .recipe_mapper import recipe_to_swift_args
from .launcher import SwiftCLILauncher
from .checkpoint import SwiftCheckpointNormalizer
from .logparser import SwiftLogParser

class SwiftAdapter(BackendAdapter):
    name = "swift"
    supported_stages = ("sft", "dpo", "kto")
    env_spec = EnvSpec(
        venv_path=Path(".venvs/swift"),
        python_executable=Path(".venvs/swift/bin/python"),
        required_packages=["ms-swift>=2.5", "transformers", "peft"],
    )

    def prepare_data(self, stream, recipe, workdir):
        out = workdir / "data.jsonl"
        write_swift_jsonl(stream, recipe, out)   # 流式写
        return out

    def prepare_config(self, recipe, data_path, workdir):
        cfg_path = workdir / "swift_config.yaml"
        recipe_to_swift_args(recipe, data_path, cfg_path)
        # 同时保存 IR 原文用于可复现
        recipe.to_yaml(workdir / "recipe.yaml")
        return cfg_path

    def build_launcher(self, config_path, recipe, workdir):
        return SwiftCLILauncher(
            env_spec=self.env_spec,
            config_path=config_path,
            recipe=recipe,
            workdir=workdir,
        )

    def checkpoint_normalizer(self):
        return SwiftCheckpointNormalizer()

    def log_parser(self):
        return SwiftLogParser()
```

`data_writer.py`、`recipe_mapper.py` 实际逻辑（细节）：

- `write_swift_jsonl`：根据 `recipe.stage` 把 `Sample` 改写成 swift 期望的字段名（`sft` 用 `messages`，`dpo` 用 `chosen/rejected`）。
- `recipe_to_swift_args`：把 `Recipe` 翻译成 swift YAML，**强制把 `target_template` 写进去**，避免默认值差异。
- `SwiftCLILauncher.command()` 返回类似 `["<venv>/bin/swift", "sft", "--config", str(config_path)]`。

## 4.2 verl adapter

`distillwheel/backends/verl/adapter.py`：

```
class VerlAdapter(BackendAdapter):
    name = "verl"
    supported_stages = ("grpo", "ppo", "rloo", "opd")
    env_spec = EnvSpec(
        venv_path=Path(".venvs/verl"),
        python_executable=Path(".venvs/verl/bin/python"),
        required_packages=["verl", "ray", "vllm"],
    )

    def prepare_data(self, stream, recipe, workdir):
        # 流式写 parquet
        from .data_writer import write_verl_parquet
        out = workdir / "prompts.parquet"
        write_verl_parquet(stream, recipe, out)
        return out

    def prepare_config(self, recipe, data_path, workdir):
        from .recipe_mapper import recipe_to_verl_overrides
        # verl 用 hydra：这里产出 override list 而非整 YAML
        overrides_path = workdir / "verl_overrides.txt"
        recipe_to_verl_overrides(recipe, data_path, overrides_path)
        recipe.to_yaml(workdir / "recipe.yaml")
        return overrides_path

    def build_launcher(self, config_path, recipe, workdir):
        from .launcher import VerlRayLauncher
        return VerlRayLauncher(
            env_spec=self.env_spec,
            overrides_path=config_path,
            recipe=recipe,
            workdir=workdir,
        )

    def checkpoint_normalizer(self):
        from .checkpoint import VerlCheckpointNormalizer
        return VerlCheckpointNormalizer()

    def log_parser(self):
        from .logparser import VerlLogParser
        return VerlLogParser()
```

**verl 特殊点**：

- `VerlRayLauncher` 启动 `ray job submit` 或直接 `python -m verl.trainer.main_ppo +overrides=...`。
- `VerlCheckpointNormalizer.normalize` 内部调用 verl 自带的 model_merger 脚本（通过子进程，在 verl venv 里跑）把 FSDP 分片合并成 HF 格式。

- **Reward function 注入**：通过 `recipe.rl.reward_fn_ref`（如 `"my_pkg.rewards:math_reward"`）把字符串引用写进 verl override，verl 在自己进程里 `importlib.import_module` 加载。约定接口签名 `(prompts, completions, **kwargs) -> list[float]`。

---

# 五、Pipeline 编排层

`distillwheel/pipeline/orchestrator.py`：

```
from pathlib import Path
from distillwheel.core.registry import load_entry_points
from distillwheel.core.router import resolve
from distillwheel.core.ir.recipe import Recipe
from distillwheel.core.ir.sample import SampleStream
from .artifacts import build_output_layout
from .preflight import run_preflight

def run_training(
    recipe: Recipe,
    stream: SampleStream,
    *,
    skip_preflight: bool = False,
) -> Path:
    load_entry_points()
    adapter = resolve(recipe)

    layout = build_output_layout(recipe.io.output_dir)
    workdir = layout.workdir

    data_path = adapter.prepare_data(stream, recipe, workdir)
    config_path = adapter.prepare_config(recipe, data_path, workdir)
    launcher = adapter.build_launcher(config_path, recipe, workdir)
    launcher.prepare_env()

    if not skip_preflight:
        run_preflight(adapter, recipe, data_path, workdir)

    parser = adapter.log_parser()
    metrics_path = layout.metrics_jsonl
    with metrics_path.open("w") as fp_metrics:
        for line in launcher.launch():
            # 1) 原始 stdout 落盘
            layout.append_raw_log(line)
            # 2) 抽指标
            m = parser.parse_line(line)
            if m is not None:
                fp_metrics.write(m.to_json() + "\n")
                fp_metrics.flush()

    if launcher.returncode != 0:
        raise RuntimeError(f"training failed rc={launcher.returncode}")
```

```
    native_dir = launcher.collect_artifacts()
    normalizer = adapter.checkpoint_normalizer()
    normalized = normalizer.normalize(
        native_dir=native_dir,
        output_dir=layout.root,
        recipe_yaml_path=workdir / "recipe.yaml",
    )
    layout.write_metadata(normalized)
    return layout.root
```

`pipeline/artifacts.py` 负责创建并维护这套目录布局：

```
{recipe.io.output_dir}/
├── workdir/                # 框架原生中间文件 (data.jsonl, config.yaml, ...)
├── raw_logs/               # 完整 stdout
├── metrics.jsonl           # 归一化指标
├── final/                  # 归一化 HF 模型
├── checkpoints/            # 归一化中间 checkpoint
├── training_recipe.yaml    # 原始 IR recipe
└── metadata.json
```

`pipeline/preflight.py`：把 `recipe.train.epochs` 改成 1 step、batch 缩到 1，跑一次 dry-run。失败提前暴露 tokenizer 不匹配、OOM 等问题。

---

# 六、扩展新框架的步骤

新增一个 backend（比如 `trl`）只需：

1. 新建 `distillwheel/backends/trl/`，实现 `TRLAdapter(BackendAdapter)` 及其 data_writer / recipe_mapper / launcher / checkpoint / logparser。
2. `__init__.py` 调用 `register_adapter(TRLAdapter)`。
3. 在 `pyproject.toml` 声明 entry point：

   ```
   [project.entry-points."distillwheel.backends"]
   trl = "distillwheel.backends.trl"
   ```

4. 准备 `.venvs/trl/`，安装 TRL 所需依赖。
5. （可选）改 `core/router.py` 的默认路由表，或让用户通过 `recipe.backend_hint="trl"` 指定。

**不需要改 `core/` 或 `pipeline/` 任何代码。**

---

# 七、跑通顺序与里程碑

按 `Distillwheel.md` 第三部分建议的最小爆炸半径推进，并适配你"先 swift 和 verl"的选择：

| 阶段 | 目标 | 验收标准 |
| --- | --- | --- |
| M0 | core/ 骨架 + 单元测试 | IR 校验、注册中心、SubprocessLauncher 用 echo 跑通 |
| M1 | swift + SFT (LoRA) | 输出符合归一化目录的 LoRA，metrics.jsonl 至少有 loss/lr |
| M2 | swift + DPO | 验证 preference 类 IR 字段 |
| M3 | verl + GRPO 最小例 | Ray launcher、parquet writer、reward_fn 注入、FSDP 分片 → HF merge |
| M4 | verl + PPO/RLOO/OPD | 复用 M3 路径，主要补 recipe_mapper |
| M5 | 注册扩展演示 | 写一个 mock backend ( `echo` 当训练) 验证 entry point 装载 |

# 八、关键设计取舍

1. `core` **不 import 任何训练框架**——`backends/*/adapter.py` 顶层也禁止，框架代码只在 launcher 启动的子进程里出现。`tests/unit` 加一条静态检查防回归。
2. **Chat template 强一致**——`recipe.target_template` 必填项 (校验阶段)，swift 和 verl 的 recipe_mapper 都把它写进各自配置；同一份模板出去，行为可比。
3. **流式 IR**——`SampleStream` 是 `Iterable[Sample]`，所有 adapter 的 `prepare_data` 接收流式而非 list，verl 的 parquet writer 用 `pyarrow.ParquetWriter` 增量写。
4. **配置可复现**——每次训练把 `recipe.yaml` 原文和翻译后的 `swift_config.yaml` / `verl_overrides.txt` 都落到 `workdir/`，metadata.json 记录 git commit。
5. **失败诊断分层**——子进程 stdout 全量落 `raw_logs/`，归一化指标落 `metrics.jsonl`；任一异常都把最后 N 行 stdout 嵌入异常消息抛出。
6. **Reward function 跨进程**——主进程不持有 reward_fn 对象，只持有引用字符串 (`"pkg.mod:fn"`)，由 verl 子进程在自己 venv 里加载，避免依赖污染。
7. **Adapter 顶层装饰器注册 vs entry_points**——同时支持：开发期可以直接 import 包来注册，生产期通过 entry_points 让 `pip install distillwheel-backend-xxx` 即插即用。
8. **多模态预留**——`Sample.images` 字段保留但 v1 不实现，避免后续 schema 破坏性变更；`recipe.meta` 字段允许 backend 私有扩展不污染主 schema。

# 附：用户视角的最小调用

```
from distillwheel.core.ir.recipe import Recipe
from distillwheel.pipeline.orchestrator import run_training
from my_data import iter_samples        # 用户自己的样本流

recipe = Recipe.from_yaml("configs/qwen_sft_lora.yaml")
out = run_training(recipe, iter_samples())
print(f"normalized model at: {out}/final")
```

或命令行：

```
distillwheel run --recipe configs/qwen_sft_lora.yaml --data data/sft.jsonl
distillwheel run --recipe configs/qwen_grpo.yaml --data data/prompts.jsonl
```

切换 backend 只需改 `recipe.backend_hint` 或调整路由表，IR 数据与目录布局完全一致。